
Five Nights at Dana User Manual

Introduction

Five Nights at Dana is a point-and-click survival game built entirely in Java. Instead of a creepy pizzeria, you're a computer science instructor working through the night in your office inside Bucknell University's Dana Engineering building. But you aren't alone — as the night goes on, determined students roam the halls desperately hunting for last-minute help with their coding assignments and projects. The goal is to make it to 6 a.m. without answering a single question, to do this you'll need to think fast: strategically lock doors, halt elevators, seal up vents, and keep a close eye on the security cameras. Can you manage the building's systems and protect your precious time, or will you be trapped answering questions all night?

Motivation

Our main motivation for creating Five Nights at Dana was to gain experience with working in a complex Java environment while creating a fun and exciting game experience. Our group all enjoys the horror genre within games and wanted to create something that was reminiscent of the franchise Five Nights at Freddy's while having it set in a familiar location like Dana Engineering.

When planning the game we looked into what kind of players we expected to enjoy the experience we were creating. This resulted in a few key user stories being: Lore Hunter, Casual Gamer, Completionist, and Game Design Appreciator. All different users helped us decide on certain design details that would cater to a broad audience. We wanted the game to be accessible and understandable to casual gamers and veterans of horror games alike while also appealing to those who seek out well designed and in-depth gameplay as well as story.

System Requirements & Execution

To play Five Nights at Dana, you just need to make sure your computer has Java 21 installed. Gradle will handle the install of the main dependencies for you. The game works on Windows, macOS, and Linux.

When you're ready to play, open your terminal or command prompt, navigate to the game's folder, and run the command that matches your operating system:

- Windows (PowerShell): `./gradlew run`
- macOS / Linux: `gradle run`

Gradle will automatically handle the setup for your specific computer and launch the game for you.

Main Menu

Once the game launches, the user is presented with the main menu. From there, they can adjust the sound using the volume sliders and begin gameplay by clicking the Start Game button. The volume sliders controls three different volume sections:

- Master Volume: Controls all the audio within the game.
- SFX Volume: Controls short sound effects that help the user keep track of the students
- Music Volume: Controls the looping ambient audio.

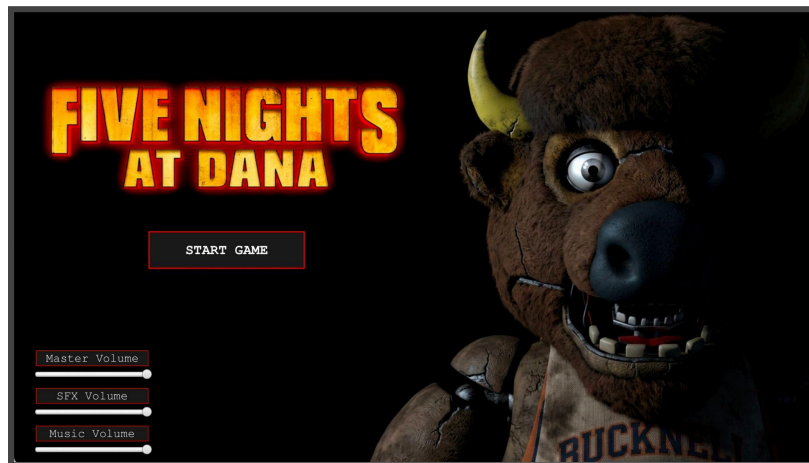


Figure 1: Main Menu Screen with volume sliders

Figure 2 shows the Use Case Diagram of our game, illustrating the player's primary gameplay interactions, survival objectives, and core system behaviors throughout each night, including monitoring cameras, managing defenses, tracking power usage, and avoiding threats in order to survive until 6 AM.

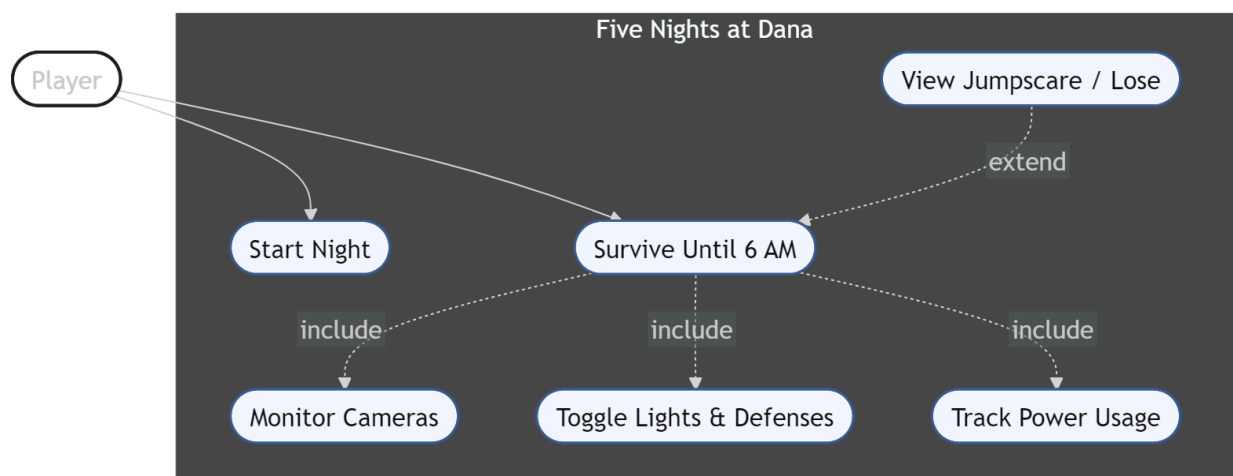


Figure 2: UML Use Case diagram for Five Nights at Dana

Gameplay & Controls

Once the night begins, the player is placed in the office view. Here, they can access tools to deter students from entering the office, monitor security cameras, and keep track of their battery life and notifications.

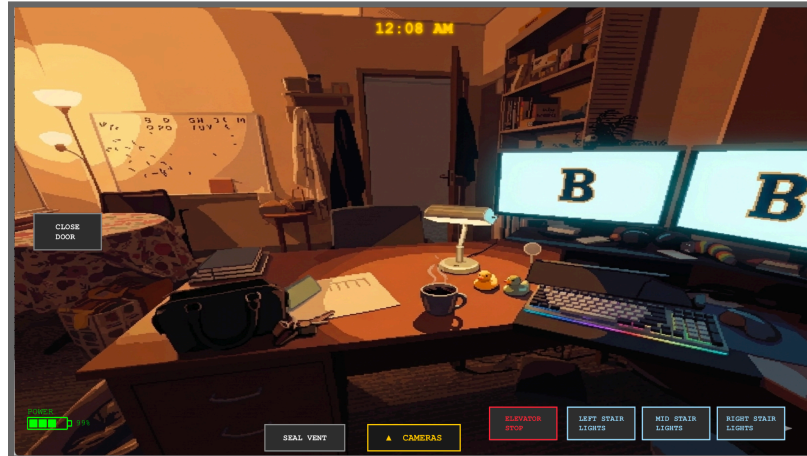


Figure 2. OfficeView Screen with UI design for player interaction

Shown below are all the different interactable features of the main menu:

- **Cameras:** Pulls up the camera system which allows the player to look around the building to keep track of the students, and to delay the running student.
- **Vents:** Vents serve as a shortcut to the players office. Only 1 student can be in the vent at a time. Once in a vent a countdown starts for them to arrive at your door. If the vent entrance in the first floor is taken, that is 15 seconds and 10 seconds for the second floor. Sealing the vent will block students from entering the vent and eject any student inside the vents. The maximum duration of a sealed vent is 8 seconds after that you have to wait out a 12 second cooldown for it to be used again. Sealing a vent drains power.
- **Door:** The door is the player's main defense mechanic. Having the door closed consumes a significant amount of power, but blocks out any student from getting to you. Use sparingly and only when necessary.
- **Elevator:** Like the vent, the elevator serves as a shortcut for students. Being able to house 1 student at a time and take them between floor 1 and 3 in a 10 second ride. Stopping the elevator consumes power and ejects the student.
- **Stair Lights:** Stairs are the main way students traverse the floors. There are no limits to how many students can be in a stairwell. The player is given 1 light charge to deter and push back students in the stairwell. Turning on the light in a stairwell consumes power and pushes back any student in the specified stairwell while its duration of 5 seconds is active. The distance a student is pushed back depends on their personality.
- **Power:** This is the main resource management aspect of the game. Optimizing power use is how the player can survive the night. Power starts at 100% and is decreased when using any system that consumes it.

Camera System

To win the game, the player must track students as they move throughout the building using the CameraView which features the Camera UI, and a 3 floor map of the building to switch between cameras.

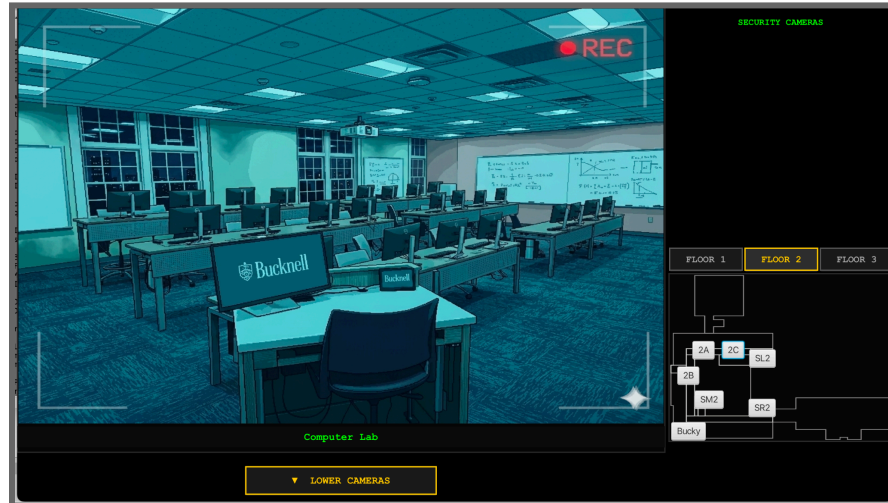


Figure 3: Second Floor Computer Lab Camera View

Shown below are all of the interactive features within the Camera View:

- **Lower Camera:** Sends the player back to the office view.
- **Switching Floors:** The player can switch between the 3 available floors using the designated button seen above the map.
- **Switching Camera feeds:** Clicking the nodes on the map switches the user to the selected camera. Cycling through the camera is imperative for locating students and for managing the runner student.
- **Computer Lab:** Located on the third floor, on camera 3D. The running student is housed. Check on him periodically to make sure he doesn't attack you. More details in the student section.

Students

There are six different student variations. Their personalities, names, difficulty levels, and preferred routes are listed below respectively:

PERSISTENT - Sean



Level: Starts at an AI level of 1 and starts to spike at 3 AM

Route: Prefers the left stairwell.

Due to his persistent nature, pushing back Sean is harder, ignoring 1 push back step when deterred through toggling the stair lights, found on the main menu.

EAGER - Faris

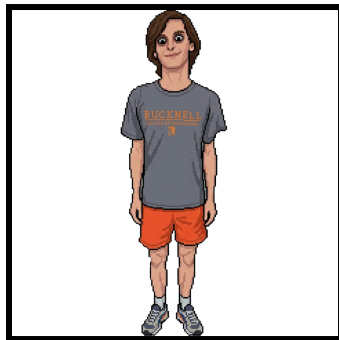


Level: Starts at an AI level of 2 and starts to increase at 2 AM.

Route: Prefers the elevator.

Due to his eager nature, Faris has a faster movement speed and prefers direct routes and shortcuts such as the elevator.

CONFUSED - Cyrus



Level: Starts at an AI level of 1 and starts to spike at 3 AM

Route: Prefers the middle stairwell.

Due to his confused nature, Cyrus has an unpredictable movement speed that changes often. He also gravitates towards the center of the building often moving into unnecessary side rooms.

SHY - Tyler



Level: Starts at an AI level of 1 and starts to spike at 3 AM.

Route: Prefers the vent system.

Due to his shy nature, Tyler has a slower movement speed and he gravitates towards the left side of the building or vent entrances.

RUNNER - Mason



Level: Inactive till 2 AM. He then starts building up his activity meter which scales up with time.

Route: Mason will only appear in Cam 3D, the computer lab, or at the door.

Activity meter: If he isn't checked on periodically using the camera system, specifically the computer lab (Cam 3D), his meters start to build. Once it fills, he will run towards the door.

INSTANT - Aiden



Level: No specific level

Route: No specific route

Aiden has a 1 in 1,000,000 chance to appear after checking the MakerE cam (Cam 1D). If Aiden appears in the MakerE camera, once you close the camera he will be in the office and you will lose the game.

Conclusion

Five Night at Dana successfully combines classic survival horror mechanics with a familiar setting to create an engaging but challenging experience. Through the implementation of multiple student types, each with unique characteristics, the game requires the player to adapt a changing strategy and manage their resources efficiently. The integration of systems such as doors, vents, elevators, stair lights, and security cameras add depth to the gameplay and encourages decision making.

From a development perspective, this project provided valuable experience in building a complex Java-based system. Overall, Five Nights at Dana achieves our goal of delivering an entertaining yet technically meaningful project that reflects both our interests in game design and allows for many users to enjoy.